Zadanie č. 1c - Travelling Salesman Problem

Ruslan Gainetdinov, ID: 127150

11. októbra 2025

Abstrakt

Problém obchodného cestujúceho (TSP, Travelling Salesman Problem) je klasickou optimalizačnou úlohou, ktorej cieľom je nájsť najkratšiu cestu prechádzajúcu cez zadanú množinu miest, pričom každé mesto musí byť navštívené práve raz a cesta sa musí skončiť v počiatočnom bode. Cieľom tejto práce je preto preskúmať a implementovať metódy, ktoré umožňujú nájsť približné riešenia v primeranom čase. V práci sú skúmané heuristické prístupy (genetic algorithm, simulated annealing). Účinnosť každého z týchto prístupov sa hodnotí na základe kvality riešenia a času výpočtu pre rôzne veľkosti úloh.

1 Genetic Algorithm

Genetický algoritmus je inšpirovaný evolučnými procesmi, ktoré sa vyskytujú v prírode. Tento algoritmus sa používa na riešenie problémov optimalizácie, pričom vytvára populáciu možných riešení a cez proces selekcie, kríženia (crossover) a mutácie generuje nové generácie riešení s cieľom nájsť optimálnu odpoveď na daný problém. V tomto prípade je genetický algoritmus použitý na riešenie problému obchodného cestujúceho (Travelling Salesman Problem, TSP), kde cieľom je nájsť najkratšiu možnú cestu.

1.1 Reprezentácia genov

Každé riešenie (jedinec) v genetickom algoritme je reprezentované ako permutácia čísel reprezentujúcich mestá. Ak máme napríklad 5 miest, jedinec môže byť reprezentovaný ako [0, 3, 4, 1, 2], čo predstavuje poradie, v akom budú mestá navštívené. Táto reprezentácia zaručuje, že každé mesto bude navštívené práve raz.

1.2 Inicializácia prvej generácie

Počiatočná populácia je vytvorená tak, že sa náhodne generujú permutácie miest. V tomto prípade je veľkosť populácie 100 jedincov (definované konštantou `POPULATION_SIZE`). Pre každý jedinec je vytvorená náhodná permutácia miest, ktorá reprezentuje možné riešenie problému. Tento proces zabezpečuje rozmanitosť riešení v prvej generácii.

1.3 Selekcia

Na selekciu jedincov pre tvorbu nových generácií sa používajú dva typy selekcie: turnajová selekcia (Tournament Selection) a selekcia pomocou rulety (Roulette Wheel Selection).

Pri turnajovej selekcii sa vyberie náhodná podmnožina populácie (turnaj) a jedinec s najlepšou fitness (najkratšou vzdialenosťou) z tejto podmnožiny je vybraný na ďalšie operácie, ako sú kríženie a mutácia. Veľkosť turnaja je stanovená na 5.

Pri selekcii pomocou rulety sa každému jedincovi priradí pravdepodobnosť výberu úmerná jeho fitness, ktorá je vypočítaná ako rozdiel medzi maximálnou vzdialenosťou v populácii a vzdialenosťou daného jedinca. Jedinci s kratšou vzdialenosťou majú vyššiu pravdepodobnosť výberu. Výber sa uskutočňuje náhodným výberom na základe týchto pravdepodobností.

1.4 Kríženie (Crossover)

Na tvorbu nových jedincov z rodičov sa používa Ordered Crossover (OX). Tento typ crossovera zachováva poradie prvkov z rodičov, čím zaisťuje, že v potomkovi sa nebudú opakovať žiadne mestá. Kríženie prebieha s pravdepodobnosťou 80 percent (definované konštantou `CROSSOVER_RATE`). Ak sa crossover neuskutoční, potomok je kópiou jedného z rodičov.

1. Vyberie sa náhodný segment miest z rodiča 1.
2. Zvyšok miest je doplnený z rodiča 2, pričom poradie miest je zachované.

1.5 Mutácia

Mutácia sa aplikuje s pravdepodobnosťou 5 percent (definované konštantou `MUTATION_RATE`). Mutácia je implementovaná ako výmena dvoch náhodne vybraných miest v permutácii, čím sa zabezpečuje, že algoritmus sa vyhne predčasnej konvergencii na lokálne minimum a bude skúmať rôzne oblasti priestoru riešení.

1.6 Tvorba novej generácie

Každá nová generácia je vytvorená tak, že sa iteratívne vyberajú rodičia z aktuálnej populácie pomocou dvoch metód selekcie: turnajovej selekcie alebo selekcie pomocou rulety. Pri turnajovej selekcii sa náhodne vyberie podmnožina jedincov (turnaj) a zvolí sa jedinec s najkratšou vzdialenosťou. Pri selekcii pomocou rulety sa rodičia vyberajú náhodne na základe pravdepodobností úmerných ich fitness, ktorá je vypočítaná ako rozdiel medzi maximálnou vzdialenosťou v populácii a vzdialenosťou daného jedinca. Potom pre každý pár rodičov sa aplikuje kríženie (crossover) a mutácia. Takto vytvorené potomstvo sa stáva novou populáciou.

1.7 Fitness funkcia

Fitness funkcia v genetickom algoritme pre TSP slúži na ohodnotenie kvality jednotlivých jedincov v populácii. V tomto prípade je fitness definovaná na základe celkovej vzdialenosti trasy, vypočítanej ako súčet euklidovských vzdialeností medzi po sebe nasledujúcimi mestami v trase, vrátane návratu do počiatočného mesta. Nižšia vzdialenosť znamená vyššiu kvalitu trasy, a teda lepšiu fitness.

Pre turnajovú selekciu sa fitness používa implicitne: jedinci sú porovnávaní priamo na základe ich celkovej vzdialenosti, pričom jedinec s najkratšou vzdialenosťou v turnaji (veľkosť turnaja = 5) je vybraný. V tomto prípade sa nevyžaduje explicitný výpočet fitness hodnoty, pretože cieľom je minimalizácia vzdialenosti.

Pre selekciu pomocou rulety je fitness funkcia explicitne definovaná ako rozdiel medzi maximálnou vzdialenosťou v populácii a vzdialenosťou daného jedinca (`max_distance - distance`). Tento prístup zabezpečuje, že jedinci s kratšími trasami majú vyššiu fitness, a teda vyššiu pravdepodobnosť výberu na kríženie. Transformácia vzdialenosti na fitness je nevyhnutná, pretože selekcia pomocou rulety vyžaduje nezáporné hodnoty fitness, ktoré sú úmerné pravdepodobnosti výberu.

Fitness funkcia tak zohráva kľúčovú úlohu pri usmerňovaní genetického algoritmu smerom k optimálnym riešeniam, kde kratšie trasy vedú k lepším výsledkom. Pre obe metódy selekcie je cieľom minimalizovať celkovú vzdialenosť, no spôsob, akým sa fitness využíva, sa líši v závislosti od mechanizmu selekcie.

1.8 Parametre genetického algoritmu

Parametre genetického algoritmu sú nastavené nasledovne:

1.9 Porovnanie metód

Porovnanie dvoch metód selekcie v genetickom algoritme pre riešenie problému komivojažéra (TSP) bolo vykonané na troch setoch miest (25, 36 a 21 miest) s parametrami: veľkosť populácie 100, 500 generácií, miera mutácie 0.05, miera kríženia 0.8 a veľkosť turnaja 5 (pre turnajovú selekciu). Každá metóda bola spustená raz pre každý set miest, čím bolo získaných šesť výsledkov.

Parameter	Hodnota
Veľkosť populácie (`POPULATION_SIZE`)	100 jedincov
Počet generácií (`GENERATIONS`)	500
Miera mutácie (`MUTATION_RATE`)	0.05 (5 %)
Miera kríženia (`CROSSOVER_RATE`)	0.8 (80 %)
Veľkosť turnaja (pre turnajovú selekciu)	5

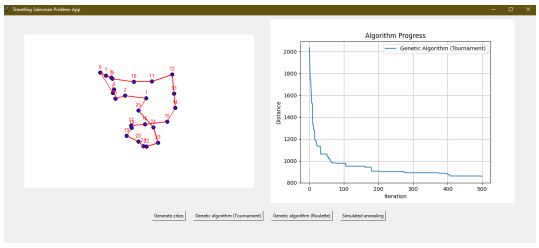
Tabuľka 1: Parametre genetického algoritmu

1. **Turnajová selekcia**: Jedinci sa vyberajú na základe najlepšej fitness (najkratšej vzdialenosti) v rámci náhodnej podmnožiny populácie (turnaj o veľkosti 5). Táto metóda rýchlo konverguje k dobrým riešeniam vďaka silnému selekčnému tlaku. Výsledky ukazujú, že dosiahla lepšie vzdialenosti vo všetkých experimentoch, s rýchlym zlepšením v prvých 200 generáciách (Obrázok 3).
2. **Roulette Wheel selekcia**: Pravdepodobnosť výberu jedinca je úmerná jeho fitness (rozdiel medzi maximálnou vzdialenosťou a vzdialenosťou jedinca). Táto metóda udržiava vyššiu diverzitu, no konverguje pomalšie a často uviazne v lokálnych optimách, ako je vidieť na priebehu konvergencie (Obrázok 4).

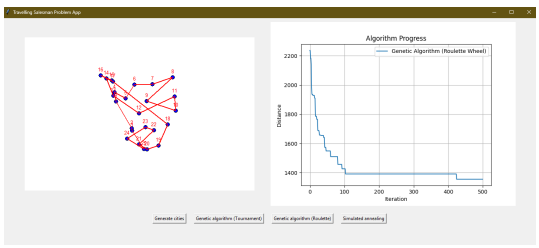
Tabuľka 4 zobrazuje priebeh najlepšej vzdialenosti pre generácie 0, 100, 200, 300, 400 a 500, spolu s finálnou najlepšou trasou pre každý experiment. Turnajová selekcia konzistentne dosiahla lepšie výsledky (napr. 798.15 pre 21 miest oproti 1191.48 pre ruletu) a rýchlejšiu konvergenciu. Roulette Wheel selekcia vykazovala pomalší pokrok a vyššie finálne vzdialenosti, pravdepodobne kvôli väčšej diverzite a nižšiemu selekčnému tlaku. Pre TSP je turnajová selekcia efektívnejšia, no ruleta by mohla byť vhodná pre problémy, kde je diverzita kľúčová.

Metóda	Počet miest	Gen. 0	Gen. 100	Gen. 200	Gen. 300	Gen. 400	Gen. 500
Turnajová selekcia	25	2035.34	978.58	907.29	892.44	884.05	858.09
Roulette Wheel selekcia	25	2236.52	1426.42	1391.04	1391.04	1391.04	1355.06
Turnajová selekcia	36	2975.84	1306.59	1234.43	1229.75	1133.90	1124.34
Roulette Wheel selekcia	36	3120.02	2124.96	2124.06	2124.06	2124.06	2124.06
Turnajová selekcia	21	1650.96	822.42	822.42	821.00	804.09	798.15
Roulette Wheel selekcia	21	1750.12	1208.22	1208.22	1208.22	1208.22	1191.48

Tabuľka 2: Porovnanie priebehu najlepšej vzdialenosti a finálnej trasy pre turnajovú a ruletovú selekciu.



Obr. 1: Konvergencia turnajovej selekcie (príklad priebehu pre 25 miest).



Obr. 2: Konvergencia Roulette Wheel selekcie (príklad priebehu pre 25 miest).

1.10 Zhodnotenie vlastností systému

Systém demonštruje efektivitu genetického algoritmu pre TSP, pričom turnajová selekcia je jednoznačne vhodnejšia pre tento problém vďaka rýchlej konvergencii a nižším vzdialenostiam. Selekcia pomocou rulety, hoci menej efektívna, ponúka väčšiu diverzitu, čo môže byť výhodné pri problémoch s viacerými globálnymi optimami.

2 Simulované žíhanie

Simulované žíhanie (Simulated Annealing, SA) je probabilistická metóda na hľadanie globálneho minima v priestore riešení. Tento algoritmus napodobňuje proces chladenia materiálov (žíhania), kde sa najprv začne s vysokou teplotou a postupne sa ochladzuje, aby sa minimalizovala energia systému. V prípade problému obchodného cestujúceho sa SA používa na hľadanie najkratšej cesty medzi mestami.

2.1 Reprezentázia údajov

Riešenie (cesta) je reprezentované ako permutácia miest, podobne ako pri genetickom algoritme. Napríklad, pre 5 miest môže riešenie vyzerať ako [0, 1, 3, 2, 4], čo predstavuje poradie miest, v akom budú navštívené.

2.2 Algorithmus

1. Počiatočné riešenie: Algoritmus začína s náhodne vytvoreným riešením (náhodná permutácia miest).
2. Počiatočná teplota: Teplota je na začiatku vysoká, čo umožňuje prijímať aj riešenia, ktoré zvyšujú celkovú vzdialenosť. Teplota je nastavená na hodnotu 10 000.
3. Iteračný proces: Algoritmus v každej iterácii vykoná malú zmenu riešenia (swap dvoch miest). Ak nové riešenie má kratšiu vzdialenosť, je prijaté. Ak je nové riešenie horšie, môže byť prijaté s pravdepodobnosťou závislou od aktuálnej teploty a rozdielu vo vzdialenostiach.
4. Zníženie teploty: Teplota sa postupne znižuje podľa definovaného chladiaceho rozvrhu (cooling schedule), ktorý je v tomto prípade definovaný ako geometrický pokles: temperature *= 0.995.
5. Koniec procesu: Algoritmus končí, keď teplota klesne pod veľmi nízku hodnotu (1e-8).

2.3 Vlastnosti systému

Algoritmus SA má niekoľko výhod, ako napríklad schopnosť prekonať lokálne minimá vďaka možnosti prijímať aj horšie riešenia na začiatku. To umožňuje lepšie preskúmať priestor riešení v počiatočných fázach algoritmu. Nevýhodou je závislosť výsledkov od správneho nastavenia počiatočnej teploty a rozvrhu chladenia.

2.4 Závislosť vlastností od rozvrhu

1. Rýchlosť ochladzovania: Pomalšie ochladzovanie (nižšia hodnota `COOLING_RATE`) umožňuje algoritmu preskúmať viac riešení, čo zvyšuje šancu na nájdenie globálneho minima. Avšak to tiež znamená dlhší čas behu algoritmu.
2. Počiatočná teplota: Vyššia počiatočná teplota umožňuje prijímať viac horších riešení na začiatku, čo zvyšuje rozmanitosť, ale môže viesť k nestabilite, ak je teplota príliš vysoká.

2.5 Parametre simulovaného žíhania

Parametre sú nastavené tak, aby umožnili dostatočné preskúmanie priestoru riešení na začiatku a postupné zameranie na kvalitné riešenia pri znižovaní teploty.

Parameter	Hodnota
Počiatočná teplota (`INITIAL_TEMPERATURE`)	10000
Konečná teplota (`FINAL_TEMPERATURE`)	10^{-8}
Miera ochladzovania (`COOLING_RATE`)	0.995

Tabuľka 3: Parametre simulovaného žíhania

Počiatočná teplota (`INITIAL_TEMPERATURE` = 10000) určuje počiatočnú pravdepodobnosť akceptovania horších riešení, čím umožňuje algoritmu široké preskúmanie priestoru riešení. Vysoká hodnota zaručuje, že algoritmus na začiatku akceptuje takmer všetky nové trasy, čo podporuje diverzitu.

Konečná teplota (`FINAL_TEMPERATURE` = 10^{-8}) definuje bod, pri ktorom algoritmus prakticky prestáva akceptovať horšie riešenia a zameriava sa na exploatáciu najlepšieho nájdeného riešenia. Nízka hodnota zabezpečuje konvergenciu k lokálnemu (alebo potenciálne globálnemu) optimu.

Miera ochladzovania (`COOLING_RATE` = 0.995) určuje rýchlosť znižovania teploty v každej iterácii (teplota sa násobí týmto koeficientom). Hodnota blízka 1 (0.995) znamená pomalé ochladzovanie, čo umožňuje algoritmu postupne prechádzať od preskúmavania k exploatácii, čím sa zvyšuje šanca na nájdenie kvalitného riešenia.

2.6 Porovnanie výsledkov s genetickým algoritmom

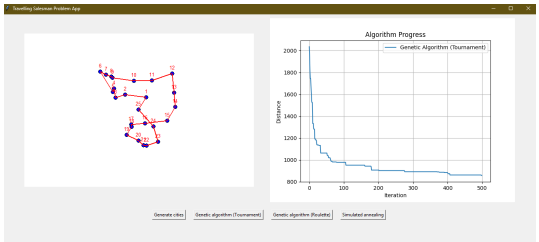
Porovnanie výsledkov simulovaného žíhania (SA) s genetickým algoritmom (GA) v dvoch variantoch selekcie (turnajová a ruleta) bolo vykonané na troch setoch miest (25, 36 a 21 miest). Každá metóda bola spustená raz pre každý set miest.

Výsledky ukazujú, že GA s turnajovou selekciou dosahuje najlepšie riešenia vo všetkých prípadoch. Simulované žíhanie je druhé, pričom často nachádza riešenia lepšie ako GA s ruletou, ale horšie ako turnajová selekcia. SA vyžaduje menej výpočtovej náročnosti ako GA s ruletou.

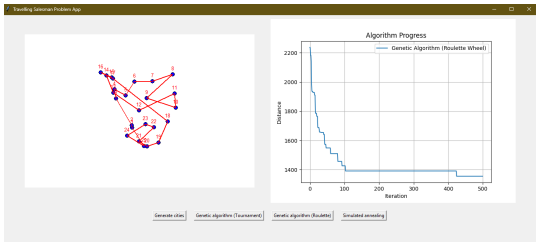
Priebeh konvergencie pre GA turnajovú selekciu je znázornený na Obrázku 3, pre ruletu na Obrázku 4 a pre SA na Obrázku 5. Tabuľka 4 zobrazuje finálne najlepšie vzdialenosti a trasy pre každú metódu a set miest. GA turnajová selekcia je efektívnejšia vďaka silnému selekčnému tlaku, zatiaľ čo SA poskytuje dobrý kompromis medzi jednoduchosťou a kvalitou riešenia. Ruleta trpí pomalou konvergenciou a vyššou variabilitou.

Metóda	Počet miest	Best route
Turnajová selekcia	25	858.09
Roulette Wheel selekcia	25	1355.06
Simulované žíhanie	25	1061.72
Turnajová selekcia	36	1124.34
Roulette Wheel selekcia	36	2124.06
Simulované žíhanie	36	1532.68
Turnajová selekcia	21	798.15
Roulette Wheel selekcia	21	1191.48
Simulované žíhanie	21	918.82

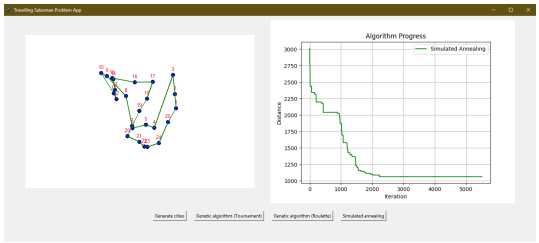
Tabuľka 4: Porovnanie finálnej trasy pre genetický algoritmus a simulované žíhanie.



Obr. 3: Konvergencia GA s turnajovou selekciou (príklad pre 25 miest).



Obr. 4: Konvergencia GA s Roulette Wheel selekciou (príklad pre 25 miest).



Obr. 5: Konvergencia simulovaného žíhania (príklad pre 25 miest).

3 Zhodnotenie riešenia a dosiahnutých výsledkov

Pri riešení problému obchodného cestujúceho (TSP) sú genetický algoritmus a simulované žíhanie dva veľmi zaujímavé prístupy, každý s vlastnými výhodami a nevýhodami. Genetický algoritmus vyniká v hľadaní globálne optimálnych riešení, najmä pri zložitejších problémoch s väčším počtom miest. Vďaka mechanizmom, ako je selekcia, kríženie a mutácia, je schopný efektívne prehľadávať priestor možných riešení a dosahovať veľmi dobré výsledky, hoci to môže trvať viac generácií.

Na druhej strane, simulované žíhanie ponúka mimoriadnu rýchlosť, ktorá sa ukazuje ako zásadná výhoda pri riešení menších a stredne veľkých problémov. Hoci v niektorých prípadoch nemusí vždy dosiahnuť tak kvalitné výsledky ako genetický algoritmus, jeho rýchlosť a jednoduchosť implementácie z neho robia atraktívnu voľbu pre časovo citlivé úlohy. Čím menej miest je v probléme, tým menší je rozdiel medzi výsledkami oboch algoritmov, a často môžu dosahovať takmer identické riešenia.

Z hľadiska praktického využitia sa preto genetický algoritmus viac hodí tam, kde je potrebné dosiahnuť najlepšie možné riešenie, aj keď je výpočet náročnejší. Simulované žíhanie však môže byť výbornou voľbou v situáciách, kde je rýchlosť kľúčová, a obetovanie malej časti presnosti je prijateľné.